

REPORT

제목: **Beautiful Soup vs Selenium: 웹 스크래핑 도구 비교**

과목명: 인공지능시스템설계

담당 교수: 이혜정 교수님

학과: 인공지능 소프트웨어학과

학번: **2603340023**

이름: 김광민

제출일: **2026. 04. 21**

1. 주요 특징 및 차이점

Selenium과 BeautifulSoup은 모두 웹 페이지를 자동화하는 데 사용되는 프레임워크입니다.

Beautiful Soup:

Beautiful Soup은 이미 받아온 HTML 코드를 분석하기 좋게 만들어주는 도구입니다. 보통 `requests` 라이브러리와 함께 사용됩니다.

엄밀히 말하면 '웹 스크래퍼'가 아니라 **HTML/XML 파서(Parser)**입니다.

- 작동 방식: 서버에 요청을 보내 받은 HTML 소스를 가져온 뒤, 원하는 태그나 클래스를 찾아 데이터를 뽑아냅니다.
- 장점: 매우 빠르고 설정이 간단합니다. 대량의 정적 페이지(뉴스, 블로그 등)를 긁어올 때 최적입니다.
- 단점: "더보기" 버튼을 눌러야 데이터가 나오거나, 스크롤을 내려야 로딩되는 JavaScript 기반의 데이터는 가져올 수 없습니다.
- 데이터 흐름: `requests` 라이브러리가 서버로부터 HTML 문서(문자열)를 다운로드하면, BeautifulSoup은 이 문자열을 트리(Tree) 구조로 변환합니다.
- 검색 방식: 트리 구조 내에서 `find()`, `select()` 등을 통해 특정 태그를 매우 빠르게 찾아냅니다.
- 특이점: 네트워크 통신과 데이터 분석이 분리되어 있습니다. 이미 내 컴퓨터에 저장된 HTML 파일을 분석할 때도 쓰입니다.

Selenium:

Selenium은 실제 크롬이나 파이어폭스 같은 웹 브라우저를 직접 띄워서 제어합니다.

본래 웹 애플리케이션 테스트를 위한 **브라우저 자동화 도구**입니다.

- 작동 방식: 브라우저를 열고, 클릭하고, 글자를 입력하고, 스크롤하는 등 사람의 행동을 흉내냅니다.
- 장점: 로그인에 필요한 사이트, 클릭해야 나타나는 정보, 무한 스크롤 페이지 등 모든 사이트에서 작동합니다.
- 단점: 실제 브라우저를 구동하므로 속도가 느리고 리소스를 많이 잡아먹습니다. 대규모 데이터를 수집할 때 효율이 떨어질 수 있습니다.
- 데이터 흐름: Python 코드가 WebDriver라는 중간 매개체를 통해 실제 설치된 브라우저(Chrome 등)에 명령을 보냅니다. 브라우저는 서버로부터 JS, 이미지, CSS를 모두 내려받아 렌더링을 완료합니다.
- 검색 방식: 브라우저가 화면에 띄운 요소(DOM) 중에서 특정 위치를 찾습니다.
- 특이점: 실제 브라우저 엔진(V8 등)을 사용하기 때문에 JavaScript가 실행된 후의 최종 결과물을 볼 수 있습니다.

2. 언제 무엇을 사용해야 하는지?

Beautiful Soup을 선택할 때

- 수집하려는 정보가 페이지 소스(Ctrl+U)에 다 들어있을 때
- 수만 개의 페이지를 빠르게 스크래핑해야 할 때
- 서버 리소스(메모리 등)가 제한적인 환경일 때

Selenium을 선택할 때

- 로그인에 필수적이고 세션 유지가 복잡할 때
- 스크롤을 내려야만 다음 내용이 보일 때
- 버튼을 클릭해야 정보가 나타날 때
- Single Page Application (SPA) 같이 JavaScript로 데이터를 불러오는 사이트일 때

3. 실무에서 부딪히는 결정적 차이

① JavaScript 렌더링 (가장 큰 차이)

- **Beautiful Soup:** 서버에서 처음 보내준 "날것의 HTML"만 봅니다. 최근 많은 사이트(React, Vue 기반)는 HTML이 비어 있고 나중에 JavaScript가 데이터를 채워넣는데, BS4는 이 "나중에 채워질 데이터"를 절대 볼 수 없습니다.
- **Selenium:** 브라우저가 직접 JS를 실행해 데이터를 채울 때까지 기다릴 수 있습니다. (WebDriverWait)

② 안티 크롤링(Anti-Crawling) 대응

- **Beautiful Soup:** 헤더(User-Agent)만 잘 설정하면 일반적인 접속처럼 보일 수 있지만, 너무 빠른 요청은 IP 차단을 유발합니다.
- **Selenium:** "실제 브라우저"를 사용하므로 탐지하기 더 어렵지만, 최근에는 Selenium 특유의 브라우저 변수(cdc_...)를 감지해 차단하는 사이트가 많습니다. 이를 피하기 위해 undetected_chromedriver 같은 별도 라이브러리를 쓰기도 합니다.

③ 리소스 및 비용

- **Beautiful Soup:** 수천 개의 페이지를 크롤링할 때 메모리 점유율이 낮고 속도가 압도적입니다.
- **Selenium:** 브라우저 프로세스 하나당 수백 MB의 RAM을 소모합니다. 멀티프로세싱(여러 개 띄우기)을 하려면 고사양 서버가 필요합니다.

4. 상세 비교 테이블

비교 항목	Beautiful Soup (BS4)	Selenium
작동 위치	메모리 상의 문자열 데이터 분석	실제 웹 브라우저 엔진 내
정 체	HTML/XML 파싱 라이브러리	브라우저 자동화 프레임워크
주요 용도	정적 웹 페이지 데이터 추출	동적 페이지 스크래핑, 웹 테스트
속도	매우 빠름 (데이터만 가져와서 해석)	상대적으로 느림 (브라우저를 띄워야 함)
JavaScript 지원	지원 불가 (서버에서 받은 HTML 만 읽음)	지원 가능 (JS 실행 후 결과 확인 가능)
리소스 사용량	낮음 (가볍고 효율적)	높음 (CPU /메모리 소모 큼)
난이도	쉬움 (초보자 권장)	보통 ~ 어려움 (웹 드라이버 설정 필요)
캡차(CAPTCHA)	대응 거의 불가능	대응 시도 가능 (직접 입력/우회)
스크린샷	불가능	웹페이지 캡처 가능

출처:

<https://beautiful-soup-4.readthedocs.io/en/latest/#>

<https://wikidocs.net/85739>

<https://wikidocs.net/137914>